

CS495 Software Project Proposal: Digital Watershed

Joel Robinson & Yohann Gouin

2026-04-17

1 Client Information

By sharing this client information and the rest of this document, you are stating that this client has provided this project as something they want (not something you created and asked if they wanted), and that they are interested in having you complete this project for your capstone.

1. Client name: Billy Friebele
2. Client title: Program Director of Studio Art
3. Client email address: wefriebele@loyola.edu
4. Client employer: Loyola University Maryland
5. How you know the client: Mr. Friebele was a previous professor that had taught me in Studio Arts.

2 Project Description

2.1 Overview

This project aims to design and create a digital system that organizes, preserves, and attempts to visualize a multi-modal body of research from the client. As the research expands, pulling in documents, maps, audio, video, 3D scans, and other materials, the variety and volume of media can become difficult to manage. The problem is the lack of a unified, and intuitive tool that can collect, tag, structure, and navigate these research threads. The purpose of the project is to create a sustainable web-based application that functions as a digital watershed, allowing the user to store items from research streams and organize them into larger conceptual categories. This container will support the clients ongoing artistic practice by providing an archive for inspiration, and also offer an interactive representation showing how individual pieces of research flow into broader themes. The client wants this project because it provides a long term stability for research workflow, a structured method for managing creative archives, and a dynamic way to display the connections alongside the artwork inspired from it. It aims to understand, explore, and communicate the relationships within the clients research.

2.2 Key Features

- Centralized Digital Archive
- Flexible Tagging & Categorization
- Item Search & Filtering
- Visualization of Tagged Relationships
- Sustainable Web Application Architecture for longevity.

2.3 Why this Project is Interesting

This project is interesting because its a mix between computer science, art, and research, making it a unique capstone that transforms a creative research process into a digital ecosystem of data for connections to happen with. The challenge of integrating multiple media types and being able to tag and scrub data from them to form relationships, and generating interactive visualizations allows for in depth work across several CS areas. The creative space between technology and the studio arts is something that is very compelling, and the client's idea of "revealing what cant be seen" through their art is something that connects with the relational tagging and visualization the product wants to do.

2.4 Areas of CS required

The subfields of computer science that are most likely to be relevant to this would be web development, because there would be frontend and backend engineering for managing data processing, authentication, API services, for the web app to function. There is also database systems, as it must store, index, categorize, and retrieve any and all media formats it holds efficiently. There is also information visualization, connecting media within the containers to visualize relationships between data, for future artistic purposes.

2.5 Potential Concerns and Questions

The aspect of the project that has me the most unsure, but is equally the most interesting, is the data visualization when the app begins to form connections via tags with the data, and the direction it should go. Is something like mind-maps of visualization too easy and should something deeper be chosen? What is the potential flourishes that could be added to make it non-generic? This is something we have never interacted with before and would be entirely new to us.

2.6 Comparison to Draft

This project is different from the projects before, as neither of us were able to get in contact with good clients before the proposal was due, and this Watershed Web App was the one option we had. This client is new, because the clients we had fielded in assignments prior were unresponsive when reaching out, which led to us looking more internally in Loyola for past professors or staff that would have needs that we could accommodate for our capstone project.

3 Requirements

3.1 Non-Functional Requirements

NFR1	Color Scheme	Visual / Design	The color scheme for the app should be white and blue.
NFR2	Password Hashing	Security	All passwords shall be hashed and salted using bcrypt.
NFR3	Error Logging	Maintainability	The system shall log errors with sufficient context to allow developers to debug or resolve issues locally.
NFR4	Recycle Bin	Reliability	The system shall support soft-deletion so items can be recovered if needed.
NFR5	Item Search Responsiveness	Reliability	Search results shall be shown progressively instead of waiting for the entire search to complete.
NFR6	Item Placeholder Logo	Visual / Design	Items without a preview shall be displayed with a standard placeholder logo.
NFR7	Item Backup	Reliability	An admin shall perform consistent backups to ensure data preservation in case of errors.
NFR8	Duplicate Detection	Reliability	The system shall detect duplicate uploads and remove them, so that archive remains clean.

Table 1: Non-Functional requirements

3.2 Functional Requirements (User Stories)

ID	Story Title	Points	Description
S1	Search Media	2	As a user, I want to search for specific media, so that I can find exactly what I need.
S2	Manage Tags	8	As a user, I want to create, read, update, and delete tags, so that I can appropriately maintain and tag research.
S3	Manage Items	8	As a user, I want to create, read, update, and delete items, so that I can maintain my data archive consistently.
S4	Media Preview	3	As a user, I want to view items with the correct preview player, so that I can consume the data within my app.
S5	Create Projects	2	As a user, I want to create projects within the web app, so that I can categorize my research items by project.
S6	Monitor Storage	2	As an admin, I want to monitor storage usage and media count, so that I can understand capacity across projects.
S7	Audit Log	5	As an admin, I want an audit log of critical actions, so that I can trace changes over time.
S8	Manage Users	8	As an admin, I want to manage users, so that the system remains secure and maintainable.
S9	Shared Links	3	As a user, I want to control visibility with shared links, so that I can share parts of my data container and projects.
S10	Recently Viewed	2	As a user, I want a recently viewed list, so that I can return to recent items or research threads.
S11	Login Logout	1	As a user, I want the capability to log in and out, so that I can access my account.
S12	External Links	2	As a user, I want to attach external links to items, so that I can reference sources without uploading files.
S13	Tag Relationships	3	As a user, I want to define relationships between tags, so that items can be related beyond basic tagging.
S14	Filter Items	3	As a user, I want to filter items by tag, category, or media type, so that I can narrow results for specific goals.
S15	Sort Results	1	As a user, I want to sort results by criteria such as newest or oldest, so that I can browse efficiently.
S16	Batch Upload	3	As a user, I want to batch upload files, so that I can ingest large collections efficiently.
S17	Timeline View	5	As a user, I want to view my research project on a timeline, so that I can understand how my ideas evolved over time.
S18	Upload Media Types	3	As a user, I want to upload various media types, so that I can preserve my full research collection in one place.
S19	Visual Connections	5	As a user, I want to see visual connections between research items and themes, so that I can understand relationships.
S20	Interactive Visualization	5	As a user, I want to interact with visualizations, so that I can explore research streams dynamically.
S21	Link Artworks	2	As a user, I want to link finished artworks to source research items, so that I can show inspiration paths.
S22	Network Graph View	2	As a user, I want to view research items as nodes in a network graph, so that I can see all connections at once.
S23	Graph Interaction	3	As a user, I want to filter, search, and interact with the network graph, so that I can navigate complex relationships.
S24	Media Metadata	5	As a user, I want to read and autofill media-specific metadata, so that each item captures relevant contextual information.
S25	Batch Tagging	2	As a user, I want to apply tags to multiple items simultaneously, so that I can organize large batches efficiently.
S26	Export Data	5	As a user, I want to export selected items and metadata, so that I can share collections or create backups.

Table 2: Functional requirements as User Stories.

ID	Story Title	Points	Description
S27	Notes	2	As a user, I want to add personal notes and annotations, so that I can record insights during research.
S28	Custom Dashboard	5	As a user, I want to customize my dashboard with widgets, so that I can quickly access relevant research areas.

Table 3: Functional requirements as User Stories.

4 System Design

4.1 Architecture

Our system will use MVC architecture. The View will be the web interface where users browse projects, search and filter, upload media, and interact with the project visualizations. The Controller will be our backend that receives requests from the UI, does authentication, input validation, and actions like item creation, tags, exporting, and graphing. The Model will represent the applications data using MongoDB, including all the different items within it, responsible for data persistence and retrieval.

4.2 Diagrams

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	95.54	85.29	89.58	96.82	
config	100	100	100	100	
db.config.js	100	100	100	100	
controllers	95.17	85.82	88.88	96.65	
authController.js	100	100	100	100	
itemController.js	90.54	76.71	77.77	93.66	51,136,207-212,265,279
projectController.js	100	100	100	100	
tagController.js	100	100	100	100	
tagRelationshipController.js	98.57	91.3	92.85	98.43	150
middleware	87.5	50	100	87.5	
upload.js	87.5	50	100	87.5	8
models	100	100	100	100	
Artwork.js	100	100	100	100	
AuditLog.js	100	100	100	100	
Export.js	100	100	100	100	
Item.js	100	100	100	100	
Project.js	100	100	100	100	
RecentView.js	100	100	100	100	
Sample.js	100	100	100	100	
Tag.js	100	100	100	100	
TagRelationship.js	100	100	100	100	
User.js	100	100	100	100	
routes	100	100	100	100	
authRoutes.js	100	100	100	100	
itemRoutes.js	100	100	100	100	
projectRoutes.js	100	100	100	100	
tagRelationshipRoutes.js	100	100	100	100	
tagRoutes.js	100	100	100	100	

Figure 1: Simplified Class Diagram

4.3 Technology

The project will be implemented as a web app using JavaScript as the programming language. The frontend will be built with React, providing an interface for browsing research items, managing projects and tags, and interaction with the visualizations. Styling will be handled using Bootstrap CSS. Data Visualizations will use JS visualizations libraries capable of rendering graphs and timelines such as recharts, d3-force, force-graph. The backend will use Node.js with express, following its MVC pattern to handle routing/validation/authentication. MongoDB with Mongoose as the object-document mapper for defining schema and structure will be used. Bcrypt will be used for hashing and salting of passwords. Jest will be used for testing, and SuperTest will work the API-level testing. We also use date-fns for date formatting and manipulation, archiver for creating ZIP archives, multer for handling of file uploads, and fs-extra for filesystem utilities.

4.4 Coding Standards

The project follows consistent coding standards to ensure readability, maintainability, and reliability throughout. All code will follow a consistent naming convention, with JavaScript variables and functions written in camelCase, React components written in PascalCase, and MongoDB models named using singular nouns while collections remain plural. API routes will follow RESTful naming conventions.

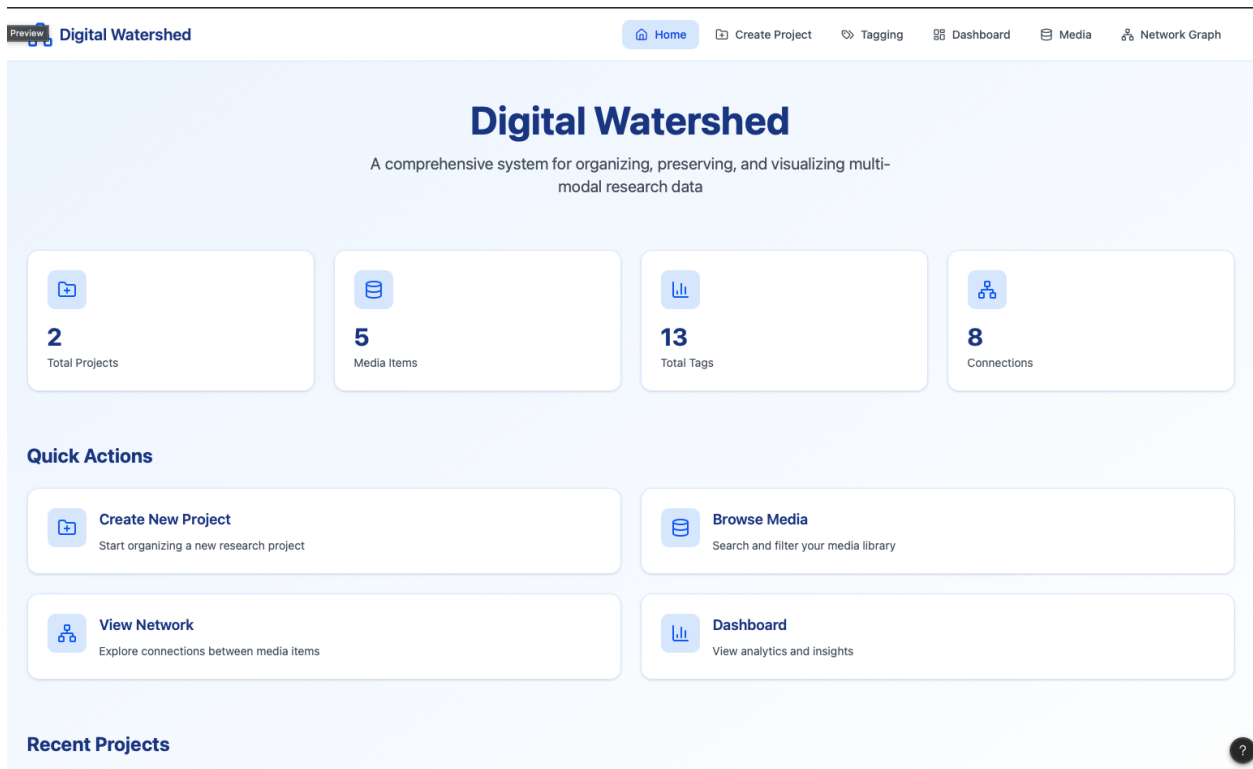
4.5 Data

Below is a high-level framework of our MongoDB collections. The system's data will be stored in MongoDB using a document-based structure that supports flexible metadata, relationships, and evolving research content.

Collection	Key Fields (examples)	Relationships / Notes
Users	_id, name, email, passwordHash, role, createdAt, updatedAt	Owns Items/Tags/Projects/Artworks; role supports Admin features. Unique index on email.
Projects	_id, ownerId, title, description, visibility, sharedLinkToken, createdAt, updatedAt	ownerId $\rightarrow$ Users._id; Items can reference multiple projects via projectIds. sharedLinkToken supports shared links.
Items	_id, ownerId, title, description, mediaType, storageType, filePath/storageUrl, externalUrl, projectIds, tagIds, metadata, notes, isDeleted, deletedAt, createdAt, updatedAt	ownerId $\rightarrow$ Users._id; projectIds $\rightarrow$ Projects._id; tagIds $\rightarrow$ Tags._id; soft-delete via isDeleted/deletedAt. Text index (title, description)
Tags	_id, ownerId, name, color, createdAt, updatedAt	ownerId $\rightarrow$ Users._id; referenced by Items.tagIds; unique compound index recommended on (ownerId, name).
TagRelationships	_id, ownerId, fromTagId, toTagId, relationshipType, createdAt	ownerId $\rightarrow$ Users._id; fromTagId/toTagId $\rightarrow$ Tags._id; supports links between tags for richer exploration/visualization.
Artworks	_id, ownerId, title, description, mediaUrl, sourceItemIds, createdAt, updatedAt	ownerId $\rightarrow$ Users._id; sourceItemIds $\rightarrow$ Items._id; enables traceability from finished work to research sources.
RecentViews	_id, userId, itemId, lastViewedAt	userId $\rightarrow$ Users._id; itemId $\rightarrow$ Items._id; supports “recently viewed” feature (S10).
AuditLogs	_id, actorUserId, actionType, targetType, targetId, timestamp, details	actorUserId $\rightarrow$ Users._id; records critical actions (S7) such as create/update/delete/login/share/export; details can store before/after snapshots.
Exports	_id, userId, itemIds, exportFormat, status, createdAt	userId $\rightarrow$ Users._id; itemIds $\rightarrow$ Items._id; tracks export requests/results (S27). Optional if exports are synchronous.

Table 4: MongoDB Schema Overview (Collections and Relationships)

4.6 UI Mocks



Homepage

Create New Project

Start organizing your research materials into a new project

Project Name *

Description *

Project Color

[Cancel](#)[Create Project](#)

?

New Project Creation Page

Tag Management

Organize and categorize your media items with tags

Media Items

Microservices Architecture Paper
3 tags

API Design Guidelines
3 tags

System Architecture Diagram
3 tags

User Interview #1
3 tags

UX Flow Mockups
3 tags



No Media Selected

Select a media item from the list to manage its tags

?

Tagging Management

Dashboard

Overview of your digital watershed

Preview

Total Projects

2

Total Media

5

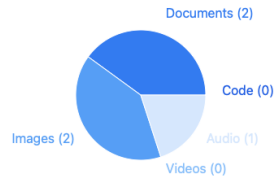
Unique Tags

13

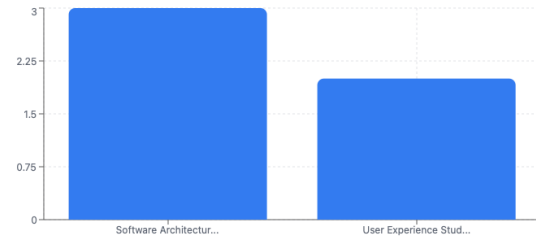
Avg Tags/Media

3.0

Media Distribution by Type



Media by Project



Top Tags

1	architecture	2
2	design-patterns	2
3	microservices	1
4	api	1
5	rest	1

Media Type Breakdown

Documents	2
Images	2
Videos	0
Audio	1

Dashboard

Media Library

Search and filter your research materials

Search media by title or description...

Filters: All Types All Projects All Tags

5 results



Microservices Architecture Paper

Document

Research paper on microservices design patterns

Software Architecture Research

architecture microservices design-patterns

Added 2/29/2024



API Design Guidelines

Document

Best practices for RESTful API design

Software Architecture Research

api rest design-patterns

Added 3/4/2024



System Architecture Diagram

Image

Visual representation of the current system architecture

Software Architecture Research

architecture diagram visualization

Added 3/9/2024



User Interview #1

Audio

Interview with senior developer about workflow challenges

User Experience Studies

interview user-research workflow

Added 3/11/2024



UX Flow Mockups

Image

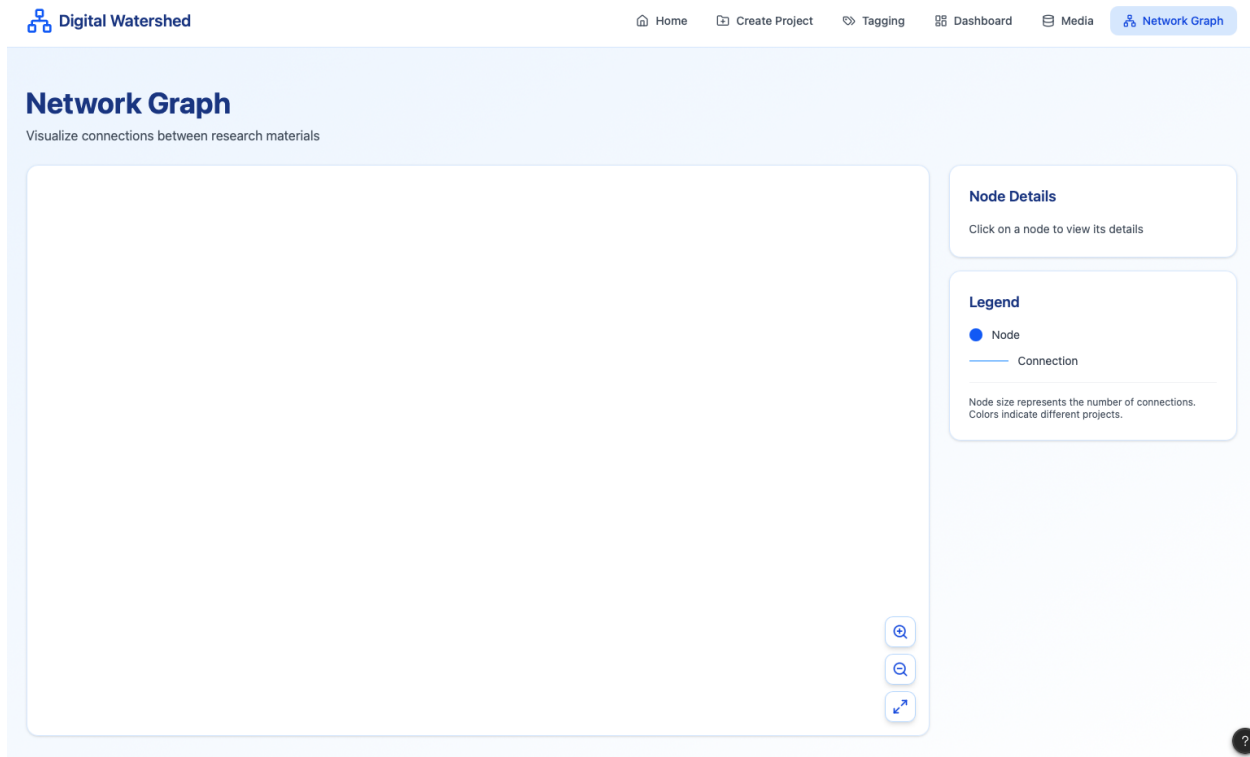
Wireframes and mockups for new user flows

User Experience Studies

ux wireframes design

Added 3/14/2024

Media Writing



Network Graph

5 Iterations

5.1 Iteration Planning

[In CS496, you plan all iterations beforehand. In CS482, you update the planning here at each iteration.]

It.	Dates	Stories	Points	
			Planned	Done
1	1/29 - 2/10	S11 Login/Logout, S3 Manage Items, S2 Manage Tags, S5 Create Projects	19	19
2	2/11 - 2/24	S18 Upload Media Types, S4 Media Preview, S1 Search Media, S14 Filter Items, S15 Sort Results, S10 Recently Viewed, S12 External Links, S27 Notes, S25 Batch Tagging	20	20
3	2/25 - 3/17	S16 Batch Upload, S24 Media Metadata, S13 Tag Relationships, S21 Link Artworks, S17 Timeline View	19	19
4	3/18 - 3/31	S19 Visual Connections, S20 Interactive Visualization, S22 Network Graph View, S23 Graph Interaction, S28 Custom Dashboard	20	20
5	4/1 - 4/14	S9 Shared Links, S26 Export Data, S6 Monitor Storage, S7 Audit Log, S8 Manage Users	23	20
Total:			95	

Table 5: Iteration Planning for Incremental Deliveries

5.2 Iteration/Sprint 1

5.2.1 Planning

This iteration plans to to complete Login and Logout (S11) (Joel), Manage Items (S3) (Joel), Manage Tags (S2) (Yohann), and Create Projects (S5) (Yohann), which is the essential CRUD functionality for the organization aspect of our clients research content. They are the pre-requisites of other user stories so they will be completed initially to allow for more intensive later iterations. This will total out to 19 points. Joel will be doing S11 and S3, while Yohann will be doing S2 and S5.

5.2.2 Work Done

User Stories S11, S3 were completely finished by Joel, while S2 and S5 were finished on the front end and the new pages created, but the hooks to the databases have not been implemented yet by Yohann. They are about 50 percent finished. These were some of the most vital functions for building upon and were completed for later iterations.

5.2.3 Testing Coverage

This coverage report is good for the first iteration, as it is around our baseline for the entire project, and has room for improvement. Index has no tests with it due to the complexity of the database connection, but will gain tests during a later iteration. itemController also doesnt have full percentage due to the deletion aspect of the CRUD not having proper testing in the file yet. Otherwise, every model and route has full coverage, and the controllers are mostly finished.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	70.11	70	60.86	70.52	
server	0	0	0	0	
index.js	0	0	0	0	10-47
server/config	100	100	100	100	
db.config.js	100	100	100	100	
server/controllers	73.5	73.91	73.33	74.13	
authController.js	100	100	100	100	
itemController.js	63.52	60	60	64.28	54-104,135-151,194-197
projectController.js	100	100	100	100	
tagController.js	100	100	100	100	
server/middleware	87.5	50	100	87.5	
upload.js	87.5	50	100	87.5	8
server/models	100	100	100	100	
Artwork.js	100	100	100	100	
AuditLog.js	100	100	100	100	
Export.js	100	100	100	100	
Item.js	100	100	100	100	
Project.js	100	100	100	100	
RecentView.js	100	100	100	100	
Sample.js	100	100	100	100	
Tag.js	100	100	100	100	
TagRelationship.js	100	100	100	100	
User.js	100	100	100	100	
server/routes	100	100	100	100	
authRoutes.js	100	100	100	100	
itemRoutes.js	100	100	100	100	
projectRoutes.js	100	100	100	100	
tagRoutes.js	100	100	100	100	

Figure 2: Coverage Report 1

5.2.4 Retrospective & Reflection

The challenges during this iteration were figuring out how we were going to store larger items in the database, and had to come across new modules to hold this. There was also the issue of starting the iteration a little late into the 2 weeks, and this is easily resolvable by just starting earlier. What we learned in this iteration was the importance of creating pages for intended features so we do not have to constantly go back and forth in page creation, and create a better workflow. Aswell as using multer for file uploads into the mongodb, which is the crux of multi media upload going forward.

5.3 Iteration/Sprint 2

5.3.1 Planning

This iteration includes Upload Media Types (S18), Media Preview (S4), Search Media (S1), Filter Items (S14), Sort Results (S15), Recently Viewed (S10), External Links (S12), Notes (S27), and Batch Tagging (S25). This iteration is about improving usability and efficiency via workflow-related user stories, allowing for better content search. This iteration is planned to deliver 20 story points.

5.3.2 Work Done

The stories completed fully this iteration were S18, S4, S1, S14, S15 by Joel, and S10 and S25 by Yohann. These stories allowed for better previewing and bulk tagging, as well as filtering through the inventory of data from our databases. What wasn't completed was S12 and S27, which were not picked up due to the complexities of S4 Media Preview and making new pages for S25 and S10. These will be pushed to Iteration 3. A total of 20 points were allotted for this iteration and 16 were fully completed.

5.3.3 Testing Coverage

This coverage is 72.89 percent of total statements, which is a very good overall score, because every model has entire coverage, and every controller except itemController is fully tested. While itemController needs to be updated for better mocking of items for the tests to complete, this will be fixed at a later date. The testing has changed in some of the controllers with changes to filepaths and mock data versus real data in these tests.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	72.89	61.72	76	74.28	
config	100	100	100	100	
db.config.js	100	100	100	100	
controllers	67.42	62.02	72.72	69	
authController.js	100	100	100	100	
itemController.js	47.7	36.17	50	49.52	54-104,133-188,228-235
projectController.js	100	100	100	100	
tagController.js	100	100	100	100	
middleware	87.5	50	100	87.5	
upload.js	87.5	50	100	87.5	8
models	100	100	100	100	
Artwork.js	100	100	100	100	
AuditLog.js	100	100	100	100	
Export.js	100	100	100	100	
Item.js	100	100	100	100	
Project.js	100	100	100	100	
RecentView.js	100	100	100	100	
Sample.js	100	100	100	100	
Tag.js	100	100	100	100	
TagRelationship.js	100	100	100	100	
User.js	100	100	100	100	
routes	100	100	100	100	
authRoutes.js	100	100	100	100	
itemRoutes.js	100	100	100	100	
projectRoutes.js	100	100	100	100	
tagRoutes.js	100	100	100	100	

Figure 3: Iteration 2 Coverage

5.3.4 Retroespective & Reflection

The pitfalls and challenges for this iteration only arose when port issues began happening with the webserver running, and the transition from mock data to real data on multiple of the .tsx files temporarily disabled them. These were fixed swiftly but took a significant amount of dev time due to Figma originally hardcoding

large portions of the data preview. Going forward we will make sure only to use real connections for data ingestion and results from it. What we learned from this was implementation of media preview and drag/drop uploading for user experience, and the ability to multi select tagging for a cleaner workflow.

5.4 Iteration/Sprint 3

5.4.1 Planning

This iteration plans Batch Upload (S16), Media Metadata (S24), Tag Relationships (S13), Link Artworks (S21), and Timeline View (S17). This allows for proper metadata ingestion for filling out many of the attributes in the schema, as well as the backbone for forming relationships for future visualizations. The total for this iteration is 19 story points.

5.4.2 Work Done

The stories completed by Joel this iteration were Batch Upload (S16), Media Metadata (S24), Tag Relationships (S13), and External Links (S12) from the previous iteration. These were the only stories completed this iteration, meaning S17 and S21 will be pushed to iteration 4. Tag Relationships is fully completed but may also be a recurring story going forward as the idea for what a "relationship" means continues to evolve.

5.4.3 Testing Coverage

The coverage from this iteration is a step up from previous iterations, mainly because the issues with previous mock tests involving the item controller have been fixed, and it was an issue with the GridFSBucket mocks for each item, but have now been squared away. We plan to hover around this coverage going forward, and if not possible, remain above 85 percent for the rest of the time.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	95.54	85.29	89.58	96.82	
config	100	100	100	100	
db.config.js	100	100	100	100	
controllers	95.17	85.82	88.88	96.65	
authController.js	100	100	100	100	
itemController.js	90.54	76.71	77.77	93.66	51,136,207-212,265,279
projectController.js	100	100	100	100	
tagController.js	100	100	100	100	
tagRelationshipController.js	98.57	91.3	92.85	98.43	150
middleware	87.5	50	100	87.5	
upload.js	87.5	50	100	87.5	8
models	100	100	100	100	
Artwork.js	100	100	100	100	
AuditLog.js	100	100	100	100	
Export.js	100	100	100	100	
Item.js	100	100	100	100	
Project.js	100	100	100	100	
RecentView.js	100	100	100	100	
Sample.js	100	100	100	100	
Tag.js	100	100	100	100	
TagRelationship.js	100	100	100	100	
User.js	100	100	100	100	
routes	100	100	100	100	
authRoutes.js	100	100	100	100	
itemRoutes.js	100	100	100	100	
projectRoutes.js	100	100	100	100	
tagRelationshipRoutes.js	100	100	100	100	
tagRoutes.js	100	100	100	100	

Figure 4: Iteration 3 Coverage

5.4.4 Retrospective & Reflection

The challenges this iteration did not come from code, but instead time management. There were major stories that were not able to get completed, like Timeline View, which will now put more stress on Sprint 4. Going forward, we need to start earlier on making progress in the iteration timeframe to make sure this doesn't happen again. What was learned this iteration were methodologies for batch upload and its functionality in gridFS, and important data that needs to exist for pages to be shared from external links, when this eventually is hosted on a non-local host platform.

5.5 Iteration/Sprint 4

5.5.1 Planning

This iteration includes Visual Connections (S19), Interactive Visualization (S20), Network Graph View (S22), Graph Interaction (S23), and Custom Dashboard (S28). These stories allow users to explore relationships through interactive visualizations and implementation of graph views for each project.

5.5.2 Work Done

The stories completed in this iteration were S28, S22, S23, and S19 by Joel. Yohann finished S17, S21, and S27 this iteration. What was not finished was S20 Interactive Visualization, and this will get moved to Iteration 5. This means that a total of 15 of 20 points have been completed, and an additional 4 points of previous iterations have been completed as well.

5.5.3 Testing Coverage

The coverage for this iteration is in the same realm as the previous coverage. Sporting an 89 percent statement coverage overall while only missing tests for the audit-log adjacent scripts, this shows room for improvement to continue gaining higher overall coverage.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	89.11	80.46	87.14	90.01	
config	100	100	100	100	
db.config.js	100	100	100	100	
controllers	88.75	81.34	87.87	89.77	
artworkController.js	90.29	89.09	100	90.8	48,99,125,143,195,217,233,261
auditLogController.js	0	0	0	0	4-91
authController.js	100	100	100	100	
itemController.js	90.19	76.71	77.77	93.19	15,55,141,216-221,274,288
noteController.js	90.76	95.34	100	89.65	35,67,86,104,135,158
projectController.js	100	100	100	100	
tagController.js	100	100	100	100	
tagRelationshipController.js	98.57	91.3	92.85	98.43	150
middleware	87.5	50	100	87.5	
upload.js	87.5	50	100	87.5	8
models	85	0	0	85	
Artwork.js	100	100	100	100	
AuditLog.js	100	100	100	100	
Export.js	100	100	100	100	
Item.js	100	100	100	100	
Note.js	57.14	0	0	57.14	54-57
Project.js	100	100	100	100	
RecentView.js	100	100	100	100	
Sample.js	100	100	100	100	
Tag.js	100	100	100	100	
TagRelationship.js	100	100	100	100	
User.js	100	100	100	100	
routes	94.73	100	100	94.73	
artworkRoutes.js	100	100	100	100	
auditLogRoutes.js	0	100	100	0	4-6
authRoutes.js	100	100	100	100	
itemRoutes.js	100	100	100	100	
noteRoutes.js	100	100	100	100	
projectRoutes.js	100	100	100	100	
tagRelationshipRoutes.js	100	100	100	100	
tagRoutes.js	100	100	100	100	

Figure 5: Iteration 4 Coverage

5.5.4 Retroespective & Reflection

The challenges this iteration held were the overall amount of work that had to be completed for this iteration while also trying to make up work not accomplished in previous iterations. The complexity of the network graphs and allowing for the interactivity proved to be more than originally thought, and it led to part of the interactivity still needing to be finished. There was previous work done, which made us feel good for completions sake. What could be done to avoid this fallback and staying consistent with our work and making sure nothing is left behind each iteration. What was learned in this iteration was how to log recently viewed media within the database, and the creation of widgets on a page how they act as technical subpages that are being viewed in the grid.

5.6 Iteration/Sprint 5

5.6.1 Planning

This iteration includes Shared Links (S9), Export Data (S26), Monitor Storage (S6), Audit Log (S7), and Manage Users (S8). These features support sharing, data preservation, admin management, and system

accountability. These functions are not part of the core functionality of the web app, so they serve as final enhancements. This iteration totals 23 story points.

5.6.2 Work Done

In this iteration, S8, S6, and S20 were completed by Joel, while S7 and S26 were completed by Yohann. What is left over from this iteration is S9. This was not completed due to the unneeded nature of the feature, which means that this user story is effectively deprecated.

5.6.3 Testing Coverage

This iteration had shown a significant decrease in statement coverage for the project. This is due to no creation of the test suites for the new files that are related to our user stories, there is still an overall 70+ percent statement coverage. Following this iteration, we will finish the test suites, due to this not changing the overall build of the code.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	72.14	61.63	70.11	72.8	
config	100	100	100	100	
db.config.js	100	100	100	100	
controllers	70.82	62.07	69.87	71.47	
adminController.js	0	0	0	0	14-258
artworkController.js	90.29	89.09	100	90.8	48,99,125,143,195,217,233,261
auditLogController.js	0	0	0	0	4-91
authController.js	100	100	100	100	
itemController.js	79.64	62.06	63.63	83.01	15,55,141,216-221,274,288,305-345
noteController.js	90.76	95.34	100	89.65	35,67,86,104,135,158
projectController.js	88.88	90	87.5	88.88	22-32
tagController.js	100	100	100	100	
tagRelationshipController.js	98.57	91.3	92.85	98.43	150
middleware	87.5	50	100	87.5	
upload.js	87.5	50	100	87.5	8
models	85	0	0	85	
Artwork.js	100	100	100	100	
AuditLog.js	100	100	100	100	
Export.js	100	100	100	100	
Item.js	100	100	100	100	
Note.js	57.14	0	0	57.14	54-57
Project.js	100	100	100	100	
RecentView.js	100	100	100	100	
Sample.js	100	100	100	100	
Tag.js	100	100	100	100	
TagRelationship.js	100	100	100	100	
User.js	100	100	100	100	
routes	79.06	100	100	79.06	
adminRoutes.js	0	100	100	0	11-26
artworkRoutes.js	100	100	100	100	
auditLogRoutes.js	0	100	100	0	4-6
authRoutes.js	100	100	100	100	
itemRoutes.js	100	100	100	100	
noteRoutes.js	100	100	100	100	
projectRoutes.js	66.66	100	100	66.66	26-27
tagRelationshipRoutes.js	100	100	100	100	
tagRoutes.js	100	100	100	100	

Figure 6: Iteration 5 Coverage

5.6.4 Retrospective & Reflection

The challenges and pitfalls of this iteration were the volume of work needing to be finished, as it was the most point heavy iteration to finish it out. This was supplemented with the removal of the Shared Links story, which allowed us to finish work of earlier iterations and still finish strong. We were not able to get out full test suites, and to avoid this in the future we need to be more efficient with our time. What was learned this iteration was the data pulling from mongodb to view the overall storage of the database being used for inventory, and the best methods for maintaining an overall audit log via archiver.

6 Final Remarks

6.1 Overall Progress

Overall, this project is in a shippable state. This project was meant to be an overall inventory container to hold our clients research media, and make projects to relate and contain these media items in a organized manner, with the ability to also make connections between research items with effective item tagging. This has all been outlined in our stories, and we have finished these implementations while also maintaining a streamlined user interface that is in accordance with our clients needs. There are things that could be tied up, but do not affect the overall vision our client had for this software. It would take less than 1 full iteration to add these features, like sharing links and external links.

6.2 Project Reflection

6.2.1 Joel's Reflection

What I have learned about the various stages of the software development process is the important of foundational building when building a web application. The process of creating the user stories for the agile development was very helpful for the forming and understanding of the important aspects of our application. This allowed us to better focus our time when it came to the core frameworks that shaped our application. The original estimate on the effort that would be required for the project certainly changed overtime. I believe there was a stark under-estimate of this project, especially in regards to the storage systems in relation to the relationship forming with tags on the items being stored, this is a very theoretical concept for how we perceived our client will create these relationships. If we were to do something differently, it would be trying to understand a more objective description of how our client wants items to connect and relate in this application, but even then, our client had changing ideas. The requirements did not change overtime, as our clients overall idea was very free-form for us to create an application from, with the core requirements staying the same for housing this data and creating projects for it. My experience with requirements collection was ranged from good to okay, it worked well for the core concepts, but when fielding specifics, our client was not versed enough to answer these questions, so we were given free reign. What I learned about the Software Design process was the importance of visual design and user interface when creating solutions to micro problems in our application. This is the aspect that the user will be interacting with, and ultimately makes up the most importance to them, aswell as its functionality. The notations that were useful in capturing design were the initial designs via figma for our front end, and how everything will fit in. I have learned many new tools as part of our software implementation. I had never user multer, archiver, recharts and more packages for graphing and interactive visualization of this data, and multer was for uploading file types into our mongoose. The experience was very good, I started by searching for different frameworks that could be used to achieve these goals, and worked with AI (Claude) to weigh these options, and find the best to implement in a smaller project like this. I then followed documentation on uses, and worked towards useful implementation. What I learned about ensuring quality is that testing my own uses are not as useful as allowing someone else to human test. There were alot of human-error bugs that were caught with someone else driving, and bug reporting back to me. Using this method, we fixed these goals and ensured quality via our other test suites above 70 percent at all times. We used jest and supertest as our testing tools for our project. My experience with jest was similar to software engineering, where it was used often and simple to create tests to cover (most) of the code, but the cases in error checking it was more difficult, and implementation for all related javascript files was an issue of volume. Supertest was simple to use for the jsx

files, and fun to learn. The steps to make sure that our code was easily maintainable is the documentation in our files, and the instructions in our github for running it. We also used little to no obfuscation, and instead opted for literal naming for it to be easier understand if it needed to be maintained further. Regarding AI usage, the AI usage in this project was to field out technologies to achieve our goals, due to the large amount of searching for packages that could achieve our goals for storage, graphing, tagging, and other more configurable tools from node.js.

6.2.2 Yohann's Reflection

This project completely changed how we think about software development and research. Going in, we thought software engineering was mostly about formulating documentation and looking up technical solutions, but working on this digital watershed system taught us it's really about understanding people and their problems. Our client works with all kinds of media: documents, maps, audio, video, 3D scans so trying to build something that actually fits into their creative workflow meant we had to spend a lot of time just listening and asking questions. The "watershed" metaphor they used to describe how research flows into bigger themes sounded cool at first, but figuring out what that actually meant in terms of buttons, databases, and user interfaces took multiple conversations and a lot of iteration. We learned that requirements aren't something you gather once at the beginning; they evolve as you build things and show them to users. The most rewarding part was definitely seeing the visualization features come together. When we finally got the interactive graph working where you could see how individual research items connected to broader themes, it felt incredible because this was solving a real problem that existing tools don't address well. It wasn't just another to-do app or CRUD project. This was something genuinely useful for creative practice. That said, the project was seriously challenging. Managing scope was brutal because the client had so many great ideas and we had to constantly balance what would make it into the MVP versus what had to wait. We also really struggled with the multi-modal media handling since every file type needs different preview players, metadata fields, and storage considerations. The technical complexity was no joke either. We built a full-stack web app with React on the frontend, implemented complex visualizations, set up media processing pipelines, and designed a flexible database schema that could handle wildly different types of content. Learning graph layout algorithms was probably the steepest learning curve, but we also had to get comfortable with media processing libraries and optimize for performance when dealing with large archives. We hit several unexpected problems along the way. The biggest surprise was how complicated 3D file formats turned out to be. We also ran into major performance issues during testing with realistic data volumes, which forced us to implement pagination, lazy loading, and better database indexing. Midway through, we realized our initial design for tag relationships was too rigid for how creative research actually works, so we added a more flexible linking system with different relationship types. And honestly, we just underestimated how long proper testing and polish would take. Looking back, here's what I'd tell anyone starting a similar project: spend way more time than feels necessary on early client requirements because assumptions you make at the beginning are expensive to fix later; build complete user flows one at a time instead of doing all backend then all frontend because integration issues surface way earlier; seriously budget time for the last 20 percent because polish and edge cases always take longer than you think; keep your documentation updated as things change because it becomes your team's shared memory; don't treat initial requirements as sacred because some of our best features came from mid-project insights; prototype the scary technical stuff early so you fail fast on hard problems; and know when to innovate versus when to just use established patterns because we saved tons of time using standard approaches for auth and CRUD while innovating on the visualization stuff. This whole experience was honestly transformative. It's one thing to build projects for class where the requirements are clear and the scope is controlled, but managing a real client with evolving needs and real constraints taught us so much about communication, adaptability, and what it actually takes to ship software that solves real problems. We're really proud of what we built and grateful for how much it pushed us to grow.

Appendix

[Appendix section if needed]